

الأتمة وحماية الجودة وحماية الفرع

Workshop 6 · Automation, Quality Gates & Branch Protection

المدة / Duration	المستوى / Level	المتطلبات / Prerequisites	المخرجات / Deliverable	المهام / Tasks	الدرجة / Points
100 دقيقة	متقدم — بعد إتمام الأوراق 01-05	بطاقة مدموجة + صلاحية في المستودع	فحص أخضر + إصدار v1.0.0 + حماية فرع	6 مهام	100 نقطة

اسم الطالب / Student	اسم المستخدم / GitHub username	المجموعة / Group	التاريخ / Date

الفكرة: اجعل الآلة تحرس المشروع بدلاً منك *The idea: let the machine guard the project*

حتى الآن كان الفحص يحدث بعد أن يرفع الطالب، ويجهد بشري. في الفرق الحقيقية، الآلة تفحص قبل الإنسان: عند كل طلب سحب يبدأ سير عمل آلي (GitHub Actions) يشغل الفحوص، وإن فشلت لا يُسمح بالدمج. النتيجة: المراجع البشري يقرأ الكود والتصميم، لا الأخطاء الميكانيكية. هذه الورقة تحوّل مشروع فريقك من «مشروع طلابي» إلى مشروع هندسي منضبط.

EN

Until now, checking happened after a push and by human effort. Real teams let the machine check first: every pull request triggers an automated workflow, and a failing check blocks the merge. Reviewers then read design, not typos.

أهداف التعلّم / Learning objectives

- فهم ما هي CI (التكامل المستمر) ولماذا لا تُقبل المشاريع المهنية بدونها.
- قراءة ملف سير عمل GitHub Actions بفهم: الأحداث (on)، المهام (jobs)، الخطوات (steps).
- تشغيل نفس الفحوص محلياً قبل الرفع بـ `node tools/ci-local.mjs`.
- اختبار الفحص نفسه: التأكد أن الفحص يفشل فعلاً عند وجود خلل (الفحص الذي لا يفشل أبداً = لا فحص).
- فهم حماية الفرع (Branch protection) و CODEOWNERS: من يراجع أي ملف، وما شروط الدمج.
- إصدار نسخة بالسوم الدلالية (SemVer) + Release على GitHub + تحديث CHANGELOG.md.

المبدأ الذي ستخرج به من هذه الورقة *The takeaway principle*

الجودة لا تُطلب بالكلام، بل تُبنى في المسار. لا تقل للفريق «افحصوا قبل الرفع»، بل اجعل الفحص شرطاً تقنياً يمنع الدمج. هذا الفرق بين فريق يعتمد على ذاكرة الناس، وفريق يعتمد على نظام.

الجزء الأول · فهم الأتمتة *Part 1 · Understanding automation*

ما هي الفحوص الآلية (CI)؟

CI اختصار Continuous Integration (التكامل المستمر): كل تعديل يُدمج تلقائياً مع فحص آلي فوري. في GitHub يُنفذ ذلك عبر GitHub Actions، وهو خادم يعمل عند أحداث محددة (فتح طلب سحب، دفع، جدول زمنية) ويشغل خطوات مرتبة تُسمى «سير عمل» (workflow).

EN

CI means Continuous Integration: every change is verified automatically. On GitHub this is GitHub Actions — a server that runs a workflow of ordered steps when chosen events occur.

1 اقرأ سير العمل في مشروعك

Read the workflow in your project

github/workflows/validate.yml.

★ 12 نقطة

متقدم

★ 15 دقيقة

الهدف: أن تعرف بالضبط ما سيحدث على الخادم عندما ترفع تعديلك.

الخطوة 1 — اقرأ الملف:

Terminal — فتح سير العمل

```
$ cat .github/workflows/validate.yml
name: validate

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  validate:
    name: فحص البطاقات والفهرس
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
      - run: node tools/build-index.mjs --check
      - run: node tests/validate-members.mjs
      - run: |
          if grep -rInE "^(<{7}={7}>{7})( |$)" --exclude-dir=.git . ; then
            echo "::error::يوجد تعارض غير محلول"
            exit 1
          fi
```

الخطوة 2 — املأ جدول التحليل:

المفتاح في الملف	معناه	القيمة عندك
name	اسم سير العمل الظاهر في تبويب Actions	
on:	الأحداث التي تُشغله	
jobs:	المهام التي تُنفَّذ	
runs-on	نظام التشغيل على خادم GitHub	
steps	الخطوات بالترتيب	عددها:
actions/checkout		—
actions/setup-node		—

لماذا يوجد خادم كامل لكل فحص؟ Why a whole server per check?

لأن GitHub يمنح كل تشغيل جهازاً جديداً نظيفاً (Ubuntu)، يستنسخ عليه مستودعك، ثم ينفذ الخطوات. هذه ميزة مهمة: النتيجة لا تتأثر بجهازك ولا بإعداداتك — «يعمل عندي» ليست حجة.

دليل المطلوب / Evidence

- الأحداث الثلاثة التي تُشغل الفحص:
- عدد الخطوات: — وما تفعله `actions/checkout` في جملة:

2 شغل الفحص نفسه محلياً

Run the very same checks locally

tools/ci-local.mjs

★ 15 نقطة

متوسط

★ 12 دقيقة

لأن انتظار الخادم يستهلك الوقت، أضفنا في المشروع محاكياً محلياً ينفذ نفس الخطوات: `node tools/ci-local.mjs`. درّب نفسك على تشغيله قبل كل `git push`.

EN
Waiting for the server wastes time, so the project ships a local mirror of the CI steps. Run it before every push.

Terminal - الفحص المحلي على فرع نظيف

```
$ git switch main
$ git pull --ff-only
$ git switch -c feature/<username>-card

# أضف بطاقتك ثم ولّد الفهرس ...
$ node tools/build-index.mjs

$ node tools/ci-local.mjs
```

=== GitHub Actions / Local CI simulation ===

- ▶ نجح ✓ ... index up to date / الفهرس محدّث
[build-index] ✓ الفهرس محدّث بالفعل.
- ▶ نجح ✓ ... member cards valid / بطاقات الأعضاء صحيحة
--- بطاقة، 0 خطأ، 0 تحذير 3 / Result: النتيجة ---
- ▶ نجح ✓ ... no conflict markers / لا علامات تعارض

✓ All checks passed / (git push) كل الفحوص نجحت - آمن للرفع ✓

اجعلها عادة واحدة One habit to build

أمر واحد يحمي سمعتك كمطوّر:

```
node tools/ci-local.mjs && git push
```

بفضل && لن يُنفذ الرفع إن فشل الفحص. هذا سطر واحد يمنع 90% من طلبات السحب الحمراء.

الصق مخرجات `node tools/ci-local.mjs` على فرعك (يجب أن تكون كلها ✓):

اذكر أسماء الفحوص الثلاثة بالترتيب:

الجزء الثاني · اكسر ثم أصلح Part 2 · Break it, then fix it

3 اكسر المشروع عن قصد وشاهد الفحص يفشل

Break it on purpose and watch the check fail

GitHub Actions

ci-local

git

★ 18 نقطة

متقدم

★ 20 دقيقة

الهدف: أن ترى بأنفسك أن الحماية تعمل فعلاً — وهذا أهم درس في الأتمتة.

قاعدة مهنية: الفحص الذي لا يفشل أبداً أسوأ من عدم وجود فحص، لأنه يمنحك ثقة زائفة. لذلك عند كتابة أي فحص، جرّبه على حالة معطوبة عن قصد أولاً، وتأكد أنه يرفضها.

EN

A check that never fails is worse than no check — it gives false confidence. Always test your check against a deliberately broken case first.

الخطوة 1 — جرّب ثلاث حالات معطوبة، واحدة واحدة:

Terminal — الحالة 1: فهرس قديم (بطاقة غير مُفهرسة)

```
$ cp data/members/00-template.json data/members/test-bad.json
# املأه سريعاً بأي بيانات صحيحة، ثم لا تولّد الفهرس
$ node tools/ci-local.mjs

فشل ✖ ... index up to date / الفهرس مُحدّث
[build-index] ✖ شغل: node tools/build-index.mjs: الفهرس غير محدّث.
فشل ✖ ... member cards valid / بطاقات الأعضاء صحيحة
✖ data/members-index.json: البطاقة "test-bad.json" غير مُفهرسة
--بطاقة, 1 خطأ, 0 تحذير 4 / Result: النتيجة ---
نجح ✓ ... no conflict markers / لا علامات تعارض

من 3 فحص فشل - لا ترفع قبل الإصلاح 2 ✖ (exit=1)
```

```
$ rm data/members/test-bad.json && node tools/build-index.mjs >/dev/null
# رسالة قصيرة، git، عدّل بطاقتك عن قصد: أمر لا يبدأ بـ
$ node tools/ci-local.mjs
```

```
❌ فشل ... member cards valid / بطاقات الأعضاء صحيحة
x omar-qa.json: اسم مستخدم غير صالح / invalid username: "Omar_QA"
⚠️ omar-qa.json: دور غير معتاد / unusual role: "سوبرمان"
x omar-qa.json: "git" الأمر المفضل يجب أن يبدأ بـ
x omar-qa.json: tags يجب أن تكون مصفوفة / tags must be an array
x sara-design.json: طول الرسالة 5 - يجب أن يكون بين 10 و140 حرفاً
--بطاقة، 5 خطأ، 2 تحذير 4 Result: النتيجة --
```

```
$ printf 'ساره\n<<<<<< HEAD\nfeature/omar-slogan\n' >
sandbox/team-slogan.txt
$ node tools/ci-local.mjs
```

```
❌ فشل ... no conflict markers / لا علامات تعارض
./sandbox/team-slogan.txt:2:<<<<<< HEAD
./sandbox/team-slogan.txt:4:=====
./sandbox/team-slogan.txt:6:>>>>>> feature/omar-slogan
الحل المقترح: ابحث عن >>>>>> واحذف العلامات واترك النتيجة الصحيحة
(exit=1) من 3 فحص فشل - لا ترفع قبل الإصلاح 1
```

درس عميق: كيف يخدعك فحص ضعيف؟ A deeper lesson: how a weak check fools you

الفحص أعلاه استخدم في الإصدار الأول من المشروع نمط بحث غير دقيق:

— ونتيجته أنه كان يمرّ بنجاح رغم وجود علامات تعارض حقيقية، لأن `grep -rInE "^(<<<<<<|>>>>>>|=====)$"`

العلامات الفعلية تكون `<<<<<< HEAD` (مع اسم الفرع بعدها)، ولأن البحث كان محدوداً بامتدادات ملفات معينة فأهمّل `sandbox/team-slogan.txt`. النتيجة: فحص يجتاز حالة معطوبة = ثقة زائفة. الدرس: اختبر فحصك على حالة مكسورة، ووسّع نطاقه ليشمل كل الملفات النصية:

```
grep -rInE "^(<{7}|={7}|>{7})($)" --exclude-dir=.git --exclude-dir=node_modules .
```

الخطوة 2 — ارفع الفرع المعطوب وافتح طلب سحب، وشاهد النتيجة على GitHub:

```
❌ All checks have failed
1 failing check
validate / فحص البطاقات والفهرس (pull_request) - Failing after 14s [Details]

❑ Merging is blocked
Required statuses must pass before merging.
```

املاً جدول الحالات الثلاث التي جرّبتها (الصق السطر الأول من كل فشل):

الحالة المعطوبة الفحص الذي فشل أول سطر من رسالة الفشل

- فهرس قديم
- بطاقة مخالفة
- علامات تعارض
- رابط تشغيل الفحص الفاشل على GitHub (Actions) :
- الصق صورة شاشة أو نص ما ظهر في «Merging is blocked» box:

4 أصلح حتى يصبح الفحص أخضر

Fix it until the check is green

git push

ci-local

★ 15 نقطة

♦ متوسط

★ 15 دقيقة

الخطوة 1 – أصلح الأخطاء الثلاثة:

الخطأ الإصلاح الصحيح

node tools/build-index.mjs ثم

git add data/members-index.json data/members-bundle.js

بطاقة مخالفة افتح الملف وصحّح الحقل المشار إليه في الرسالة، ثم أعد الفحص

git add احذف الأسطر الثلاثة للعلامات ودمج النص الصحيح، ثم

علامات
تعارض

Terminal - الفحص بعد الإصلاح

```
$ node tools/build-index.mjs
[build-index] ✓ بطاقة 4 فهرس: 4
  • ali-h مطوّر

$ node tools/ci-local.mjs
▶ الفهرس مُحدّث ... ✓ نجح
▶ بطاقات الأعضاء صحيحة ... ✓ نجح
▶ لا علامات تعارض ... ✓ نجح
✓ كل الفحوص نجحت - آمن للرفع (git push) / All checks passed. (exit=0)

$ git add -A
$ git commit -m "fix(members): correct card fields and regenerate index"
$ git push
89f5cff..3c7d9e1 feature/omar-qa-card → feature/omar-qa-card
```

الخطوة 2 – تحقق من النتيجة على GitHub:

- ✓ All checks have passed
 - 1 successful check
 - validate / فحص البطاقات والفهرس (pull_request) - Successful in 12s [Details]
- This branch has no conflicts with the base branch.
 - Merging can be performed automatically.

مهارة تُقيّم عليها *A graded skill*

لاحظ أن الفحص يحمل نفس الاسم قبل الإصلاح وبعده (فحص البطاقات والفهرس / validate)، لكن حالته تغيّرت من ✗ إلى ✓. لذلك لا يُقبل في المشاريع المهنية قول «الفحص كان يعطي خطأ» بلا لقطة أو رابط — الدليل هو تبويب Checks في طلب السحب.

دليل المطلوب / Evidence

- الصق مخرجات `node tools/ci-local.mjs` بعد الإصلاح (كل الفحوص ✓):

- رابط الفحص الناجح على GitHub:
- كم استغرق الفحص على الخادم؟ ثانية — وعلى جهازك؟ ثانية

الجزء الثالث · حماية الفرع والإصدارات *Part 3 · Protection & releases*

5 حماية الفرع وملّك الكود

Branch protection and code owners

CODEOWNERS

Settings

★ 15 نقطة

متقدم ♦

★ 18 دقيقة

حماية الفرع تعني: «لا يمكن الكتابة في main إلا عبر طلب سحب مستوفٍ للشروط». أما CODEOWNERS فيعني: «هذا الملف لا يُعدّل بلا مراجعة صاحبه». معاً يحوّلان الاتفاقات الشفهية إلى قواعد يفرضها GitHub نفسه.

EN

Branch protection turns agreements into enforced rules: no direct writes to main, only PRs that satisfy the conditions. CODEOWNERS routes each path to the right reviewer.

الخطوة 1 — افحص قواعد الحماية المطلوبة:

GitHub - Settings ← Branches ← main

- ✓ Require a pull request before merging
 - ✓ Require approvals: 1
 - ✓ Dismiss stale pull request approvals when new commits are pushed
- ✓ Require status checks to pass before merging
 - فحص البطاقات والفهرس / validate: الفحص المطلوب
- ✓ Require conversation resolution before merging
- ✓ Do not allow bypassing the above settings
 - ☐ Allow force pushes ← يجب أن يبقى معطّلاً دائماً
 - ☐ Allow deletions ← يبقى معطّلاً

Terminal - محتوى ملف مَلَك الكود

```
$ cat .github/CODEOWNERS
# الملفات المشتركة الحساسة → قائد الفريق
/data/site-config.json      @you-instructor
/js/                        @you-instructor
/css/                      @you-instructor
/tools/                    @you-instructor
/tests/                    @you-instructor
/.github/                  @you-instructor
/docs/                     @you-instructor
*                           @you-instructor
/data/members/             @you-instructor
```

جَرِّب عملياً: افتح فرعاً وعدّل ملفاً مشتركاً (`js/app.js`) مثلاً — تعديل تعليق فقط، ثم ارفعه وافتح طلب سحب، ولاحظ أن GitHub أضاف المراجع المذكور تلقائياً.

GitHub - ما يظهر في طلب السحب

Reviewers
@you-instructor - Awaiting review (requested automatically by CODEOWNERS)

من يراجعهِ إلزامياً؟ لماذا؟

الملف / المسار

.....	js/ و css/
.....	data/members/<username>.json
.....	data/site-config.json
.....	tests/ و tools/

لا تُعْظِل الحماية لإكمال عملك *Never disable protection to get unblocked*

أشهر خطأ: طالب عالق الفحص عنده فيعْظِل حماية `main` «مؤقتاً» ثم ينسى. النتيجة: مشروع بلا حماية ودين تقني أمني. الحل الصحيح: أصلح السبب، أو استخدم استثناءً موثقاً من المدرب (Bypass) يُسجّل في سجل المستودع ويُعاد فوراً.

دليل المطلوب / Evidence

- هل حماية الفرع مفعّلة على `main` ؟ ☐ نعم ☐ لا
- الفحوص المطلوبة للدمج (Required checks):
- من يراجع الملفات المشتركة حسب `CODEOWNERS` ؟
- الصق لقطة/وصفاً لطلب سحب ظهر فيه المراجع مضافاً تلقائياً:

6 أصدر نسخة v1.0.0: وسم + Release + سجل تغييرات

Release v1.0.0: tag, release notes, changelog

CHANGELOG

GitHub Releases

git tag

★ 15 نقطة

متقدم

★ 15 دقيقة

الخطوة 1 — أنشئ وسم الإصدار (Annotated tag):

```

$ git switch main
$ git pull --ff-only
$ git log --oneline -4
e20b664 (HEAD → main) Merge pull request #32 from ali-h/feature/ali-card
eb1df06 feat(members): add card for ali-h
388e173 feat(members): add card for leen-docs
ccbdca2 chore: initial commit of class-team-hub

$ git tag -a v1.0.0 -m "الإصدار الأول: موقع دليل فريق الصف مع 4 بطاقات وسير تحقق تلقائي"
$ git tag -l
v1.0.0

$ git describe --tags
v1.0.0

$ git push origin main
Everything up-to-date

$ git push origin v1.0.0
To https://github.com/school-git-workshop/class-team-hub.git
 * [new tag]          v1.0.0 → v1.0.0

```

الوسوم لا تُرفع مع الفروع! Tags are not pushed with branches

`git push` يرفع الالتزامات فقط. الـوسـم يحتاج `git push origin v1.0.0` أو `git push origin --tags` لرفع كل الـوسـوم. وهذا أشهر سبب لسؤال: «أنشأت الـوسـم ولا يظهر على GitHub».

الخطوة 2 – أنشئ Release على GitHub:

Releases ← Draft a new release
 Choose a tag: v1.0.0 Target: main
 Release title: v1.0.0 – دليل فريق الصف
 Describe this release (ملاحظات الإصدار):

الإضافات

- موقع دليل فريق الصف مع بطاقة لكل عضو
- فحص تلقائي للبطاقات والفهرس عند كل طلب سحب
- (CODEOWNERS) وملأ الكود main حماية فرع

الإصلاحات

- تصحيح نطاق فحص علامات التعارض ليشمل كل الملفات النصية

ملاحظات الترقية

- لا يوجد تغيير يكسر التوافق

الخطوة 3 – أنشئ وسم إصدار ثانٍ ثم افحص الفرق:

```
$ git tag -a v1.1.0 -m "ميزة الفلتر بالوسوم"
$ git tag -n1
v1.0.0      الإصدار الأول: موقع دليل فريق الصف مع 4 بطاقات وسير تحقق تلقائي
v1.1.0      ميزة الفلتر بالوسوم

$ git push origin v1.1.0
* [new tag]          v1.1.0 -> v1.1.0

$ git ls-remote --tags origin
1105c78548ddca65f03d9ea10f58ecddd0d15      refs/tags/v1.0.0
e20b6644b539bbb22200dde76ea5e5015d826b86      refs/tags/v1.0.0^{}
```

أكمل **CHANGELOG.md** : انقل ما تحت [Unreleased] إلى قسم الإصدار بتاريخه، وابدأ قسماً جديداً فارغاً. هذا ما يفعله أي فريق محترف عند كل إصدار.

دليل المطلوب / Evidence

- اسم الإصدار: `v.....` — رابط الإصدار على GitHub: `.....`
- الصق مخرجات `git tag -n1` :

```
$ git tag -n1
```

- الصق قسم الإصدار من **CHANGELOG.md** بعد التحديث: `.....`
- هل ظهر الوسم على GitHub بعد الدفع؟ ☐ نعم ☐ لا — وإن لا، فما الأمر الذي كان ناقصاً؟ `.....`

أسئلة تحقّق ختامية / Final knowledge checks

س1. ما الفرق بين «حماية الفرع» و«فحص CI» من حيث ما يمنعه كل واحد منهما؟

إجابتك / Your answer

س2. طالب قال: «الفحص يمرّ دائماً، إذاً مشروعنا سليم». ما الخلل المنطقي في عبارته؟ وكيف تختبر الفحص؟

إجابتك / Your answer

س3. ما الفرق بين وسم بسيط (`git tag v1.0.0`) ووسم مع تعليق (`git tag -a v1.0.0 -m "..."`)؟ وأيُّهما يُستحسن للإصدارات؟

إجابتك / Your answer

س4. ما فائدة أن يمنع المشروع «الدفع القسري» (Allow force pushes) على `main` ؟

إجابتك / Your answer

قائمة الفحص النهائية / Final checklist

- ☐ [] قرأت `validate.yml` وأعرف أحداثه وعدد خطواته
- ☒ [] شغلت `node tools/ci-local.mjs` قبل الرفع وأصبحت كل الفحوص ✓
- ☐ [] جرّبت ثلاث حالات معطوبة وتأكدت أن الفحص يفشل فيها فعلاً
- ☐ [] أصلحت الأخطاء حتى صار الفحص أخضر على GitHub (وبرابط دليل)
- ☐ [] حماية الفرع مفعّلة، والفحص المطلوب للدمج محدّد
- ☐ [] `CODEOWNERS` مطبّق، ولاحظت المراجع يُضاف تلقائياً
- ☐ [] أصدرت `v1.0.0` مع ملاحظات إصدار، وحُدّثت `CHANGELOG.md`

تقييم ذاتي للوحدة المتقدمة / Self-assessment

4
أتقنت

3
أنجزت

2
بدأت

1
لم أبدأ

بطاقة التسليم / Submission card

القيمة

البند

.....

رابط تشغيل ناجح في Actions

.....

رابط تشغيل فاشل (الحالة المعطوبة)

.....

رابط الإصدار

هل أنشأت فرعاً للتجربة (تجربة الجودة)؟ ☐ نعم ☐ لا

أصعب فحص فهمته ولماذا؟

الوقت المستغرق

ما بعد الأتمتة؟ / What comes after automation

أصبح مشروعك الآن: بطاقات موثقة، وفحص آلي، وحماية فرع، وإصدار مرّقم. الخطوة التالية مهنيّاً هي **المراقبة والتحسين**: قياس زمن الفحص، وتقليل تكرار الأعطال بنوع واحد (مثل خطأ JSON)، وإضافة فحص جديد لكل عطل يتكرر مرتين — هذا هو جوهر التحسين المستمر (Continuous Improvement).

✓ أتممت الوحدة المتقدمة — مستوى عمل الفريق الهندسي